JavaScript

What is JavaScript?

- It is both scripting and programming language.used to add functionality to html element in frontend and also used to processing action in backend.
- It can be used for both frontend and backend development.
- It is a scripting language for frontend helps to add functionality/behavior of html element in web browser.
- It is programming language for backend it processes the action using nodejs.
- JavaScript was invented by Brendan Eich in 1995.

Role of JS in Web Development:

- It is used to add functionality and interactivity to the web page.
- It is a light weight and object based programming language.
- Java Script can be directly embedded to HTML page.
- JavaScript is an interpreted language.
- It is an open-source language.
- JavaScript is case sensitive language.

Where JS Runs?

- JavaScript is an interpreted language, meaning it is executed by a JavaScript engine inside browsers (or outside using Node.js).
- Different browsers use different JS engines like Firefox uses SpiderMonkey, Chrome uses V8, Safari uses JavaScriptCore, Microsoft Edge uses Chakra
- A JS engine has the following key parts: Parser, Compiler (JIT – Just In Time), Memory Heap and Call Stack.
- The parser reads your JS code line by line, checks for syntax errors.If everything is valid, it passes code to the compiler.
- The compiler converts JS code into bytecode.
- The memory heap stores **objects, variables, functions**, and all declared entities in JS.
- The class stack executes functions **in order**, using a stack structure (LIFO – Last In First Out).Manages function calls and tracks which operation is currently running.

.JS->browser->JS engine-> parser(validation)->compiler(JIT)->heap memory->call stack->o/p

- Normally, JS runs inside browsers, giving functionality to HTML elements (e.g., buttons, forms).
- **Node.js** allows JS to run outside the browser (server-side):
    - Uses V8 engine.
    - Enables building backend apps, scripts, CLI tools, etc.
    - No browser needed.

Adding JS to HTML:

1. Internal js:
    - JS can be added within the html doc either in head or body section using script tag
    - Syntax:

```
<head>
    <script>
        //js code
    </script>
</head>
   or
<body>
    <script>
        //js code
    </script>
</body> External js
```

    - In html we can add the JavaScript using the script tag in head tag or the body tag.

      <script>…</script**>**

2. External js:
    - We make a separate file for js with .js extensions
    - To connect html with js we use script with src attribute
    - Eg:

      <script src=“index.js”></script>

3. Inline js:
    - Inline JavaScript is written inside HTML tags to add simple functionality or handle events.
    - <button onclick="alert('Button clicked')">Click Me</button>

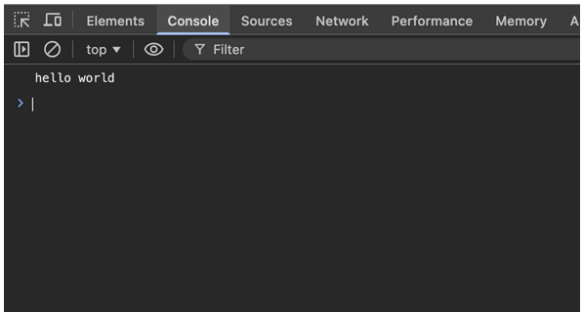
Console & Debugging:

- Browsers have a developer console which helps in testing or debugging purpose for users
- The console is a tool available in the browser’s Developer Tools that helps developers test, monitor, and debug JavaScript code.

Javascript output and input statements:

1. Output Statements:
   - Output statements are used to display information to the user.

i. console.log():
- Used to display output in the browser's developer console.
- Mainly used for testing and debugging programs.



ii. document.write():
- Used to display content directly on the web page.
- Writes HTML or text into the document.

iii. alert() or window.alert():
- Displays a popup message box on the screen.
- Contains an OK button.
- The rest of the page loads only after clicking OK.

iv. Dom method

- document.getElementById(‘’): Used to access and manipulate HTML elements dynamically using JavaScript. The method selects an element using its id.

```
frontend-assign > JSS.html > html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <h1>Hello</h1>
    <p id='para'> </p>
    <script>
       document.getElementById('para').textContent="Banglore"
     </script>
</body>
</html>
```

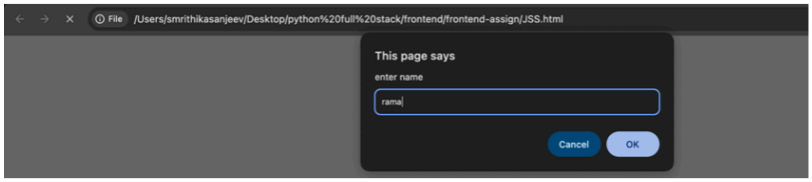
- innerHTML(), textContent() are methods used with DOM

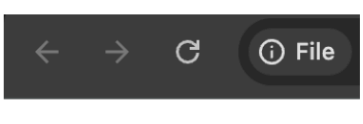
2. Input Statements:
   - Input statements are used to take input from the user.

i. prompt():

- Used to take input from the user through a popup window.
- The prompt displays as a popup window the parameter is displayed on top below that we will have single line text field to add data. 2 buttons are present cancel and ok.Once we add data and click ok the value is returned to the variable and if we click on cancel null is returned to the variable

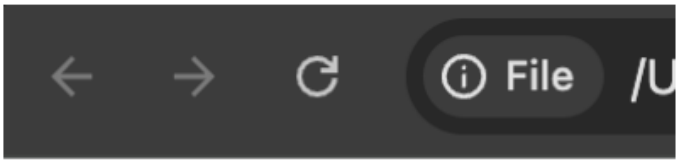
```
frontend-assign > JSS.html > html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <script>
       var name=prompt("enter name")
       document.write(name)
    </script>
</body>
</html>
```





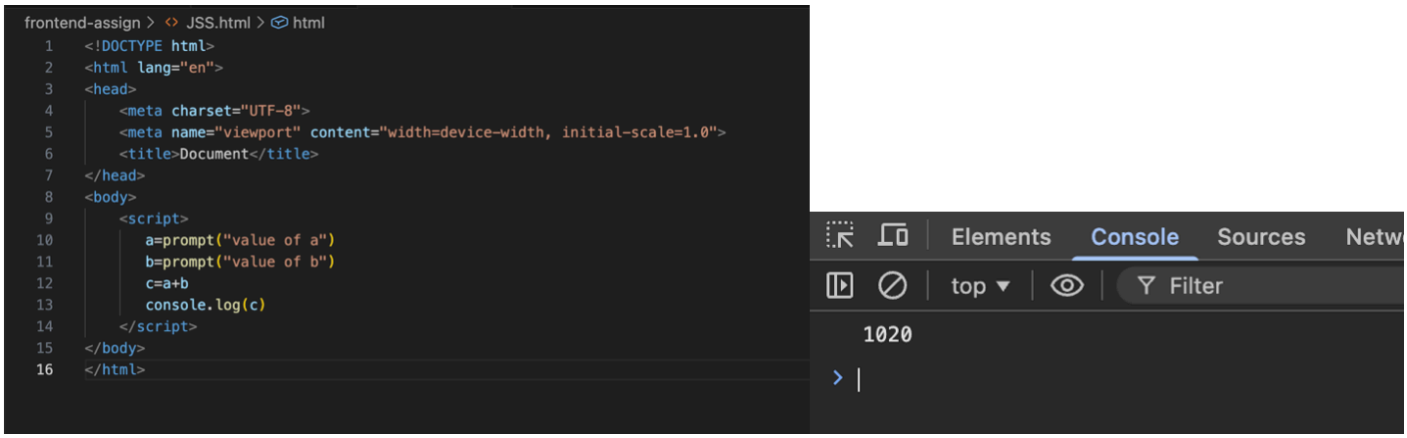
rama

- If a user clicks on cancel it returns null



null

- Eg: Take 2 values from user and display the concatenated content



JS Variables:

- Containers that store values/data in the memory
- Helps to store, use and perform some operation with data
- Keyword variable_name=value;
- Keywords are let,

- Rules for variable declaration and naming:

1. Variable naming must start with a letter or $ or _ (underscore)
   Eg: name="abc" or $name="abc" or _name="abc"
2. We cant start variable naming with number or with any other special character except $ and _
   Eg: %name="abc" or 1name="abc"
3. We can have number/other special character after first letter of variable name
   Eg: n1ame or name1 or name%
4. We cant have space in variable name
   Eg: Person name="abc"
5. We cant take any predefined keywords in JS as variable name
   Keywords like this, finally, try, catch cant be used eg: this="abc"
6. Always variable in JS should declare through variable key words
   Eg: variable key words like var, const, let eg: let name="abc"
7. It is case sensitive

Variable keywords in JS:

- It is used to declare variable in JS. var, let and const

i. var:

- **It is a legacy key word in js (used from the 1st version of js to declare variable)**
- Used to give functional scope to the variable
- Can be redeclared and can also be reinitialized
- Eg:

```
function demo() {
    var a=10
    console.log(a)//10
    {
        console.log(a)//10
    }
}
// console.log(a) //error as we cant access it outside a function
demo()
```

```
var a=10
console.log(a) //10
var a=20 //redeclaration
console.log(a)//20
a=30 //reinitialization
console.log(a)//30
```

ii. let:

- Introduced in ES6 version of JS
- It has a block scope
- let can be reinitialized but cant be redeclared
- Eg:

```
function demo() {
    {
        var a=10
        console.log(a) //10
    }

    console.log(a) //error as its outside the block even when inside the

    }
demo()
console.log(a) //error as its outside the block
```

```
let a=10
console.log(a) //10
let a=20 //redeclaration
console.log(a) //error
a=30 //reinitialization
console.log(a)//30
```

iii. const:

- Introduced in es6 version
- It has a block scope
- Const cant be redeclared or reinitialized
- Eg:

```
function demo() {
    {
    const pi=3.14
    console.log(pi) //3.14
    }
console.log(pi)//error
}
demo()
console.log(pi)//error
```

```
const a=10
console.log(a) //10
const a=20 //redeclaration
console.log(a) //error
a=30 //reinitialization
console.log(a)//error
```

Feature	`var`	`let`	`const`
Scope	Function	Block	Block
Redeclaration	Allowed	Not allowed	Not allowed
Update Value	Allowed	Allowed	Not allowed
Introduced	Old JS	ES6	ES6

Scope in JavaScript:

- Scope refers to the area or region of a program where a variable is accessible
- It determines where variables and functions can be accessed in the code.

Types of Scope in JavaScript:

1. Global Scope:A variable declared outside any function or block has global scope. It can be accessed anywhere in the program.

Eg:

```
1   let city = "Bangalore";
2
3   function showCity() {
4       console.log(city);
5   }
6
7   showCity();
8   console.log(city);
```

```
Bangalore
Bangalore
```

The variable city is accessible inside and outside the function.

2. Local Scope or Function Scope:Variables declared inside a function are accessible only inside that function.

Eg:

```
function functionscope() {
    let num = 10;
    console.log(num);//10
}

functionscope();
console.log(num); // Error
```

3. Block Scope:Variables declared are accessible only within that block.
Eg:

```
function demo(){
    {
        var a = 10;   // declared with var
        console.log(a);  // 10 (inside block)
    }
    console.log(a);  // 10 (inside function)
}
demo()
console.log(a);  // Error (outside function)
```

4. Lexical Scope: It occurs in a nested functions in JS. An outer function variable can be accessed inside the inner function but inner function variable cant be accessed in the outer function.
   It can be declared with var, let and const variable type

Eg:

```
function outer(){
    var a = 10; // outer function variable
    return function inner(){
        var b = 20; // inner function variable
        console.log(a); // 10
        console.log(b); // 20
    }
}
let r = outer();  // outer returns the inner function
r(); // call the inner function
```

Data types in JS:

- It defines the type of value a variable can store in the memory.
- JS is a dynamically typed language we don't need to specify the data type during declaration the compiler will understand the data type based on the value. But we need data type when we need to perform type conversion.
- Categories of data types:

1. Primitive data type:
   - Primitive data types store actual values and represent single, it is a basic datatype.
   - Store a single value
   - Immutable (their value cannot be changed, only replaced)
   - Number, string, boolean, null, undefined, symbol, big int

   1. Number type: integers, decimal , float comes under number data type (e.g., 10, 3.14)
   2. String : Text values (e.g., "Hello")
   3. Boolean type: variable that contains value true or false
   4. Null – Represents intentional empty value ie declaring a variable and storing null value is called null data type
   5. Undefined – Variable declared but not assigned
   6. Symbol – Unique identifier (introduced in ES6) created using symbol() constructor
   7. BigInt – Large integers beyond the Number limit

```
let num1=24
console.log(num1)
var num2=34.2
console.log(num2)
const num3=-12
console.log(num3)
let num4=-12.4
console.log(num4)

let name="abc"
console.log(name)
var city='bangalore'
console.log(city)
const age="25"//considered as string
console.log(age)
let isplaced="true"
console.log(isplaced)

var city
console.log(city)

```

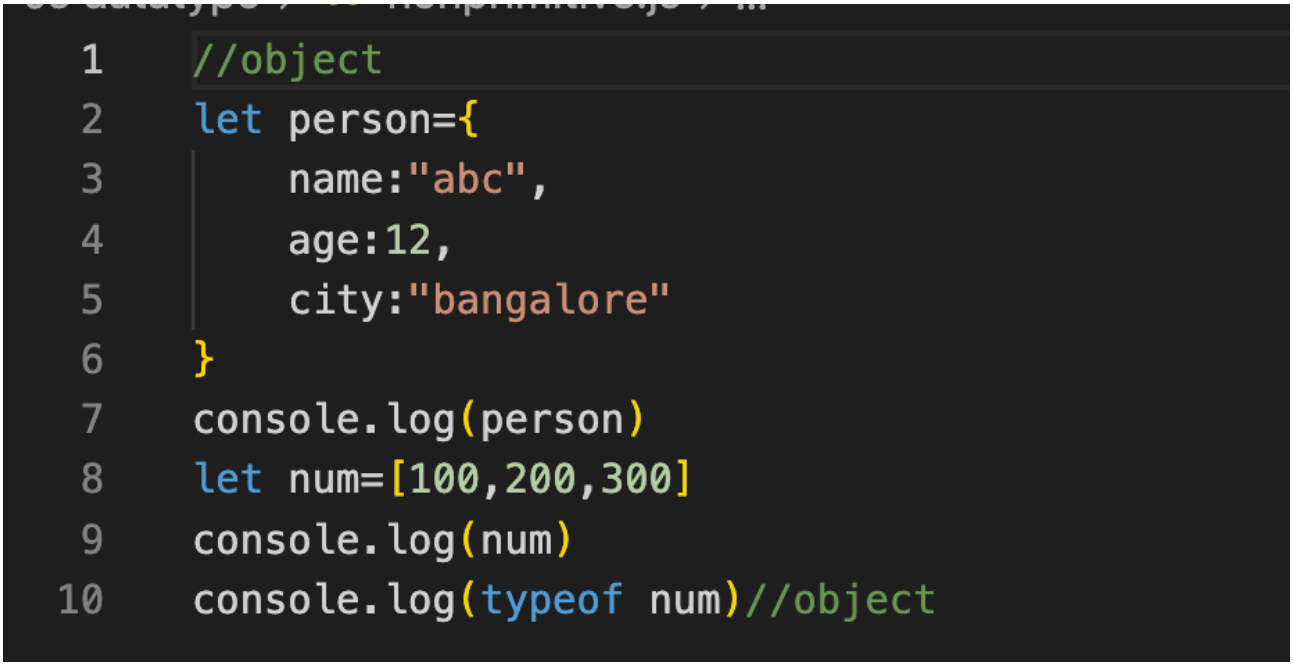
2. Non- Primitive data type:
    - Non-primitive types store collections of data or complex structures.
    - Can store multiple values
    - Mutable (values can be changed)
    - Variables store a reference (address) to the object
    - Object, Array, Function

Operators:

- Symbols used to perform operations on operands

1. Arithmetic Operators:
    - Arithmetic operators are used to perform arithmetic between variables and/or values.

Operator	Description
+	addition
-	subtraction
*	multiplication
/	division
%	modulus
++	increment
- -	decrement



```
1   //object
2   let person={
3       name:"abc",
4       age:12,
5       city:"bangalore"
6   }
7   console.log(person)
8   let num=[100,200,300]
9   console.log(num)
10  console.log(typeof num)//object
```

2.Assignment Operators:

- Assignment operators are used to assign values to variables

Operator	Description
=	Assign
+=	Add and assign
-=	Sub and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulus and assign

Comparison Operators:

- Comparison operators are used in logical statements to determine equality or difference between variables or values.

Operation	Description
==	Is equal to
===	Is exactly equal to (value and type)
!=	Is not equal to
>	Greater than
<	Less than

Logical Operators:

- Logical operators are used to determine the logic between variables or values.

Operation	Description
&&	And
||	Or
!	not

Special Operators:

Operation	Description
?:	Conditional
,	Comma. Multiple expressions as single statement
delete	delete property from an object
in	checks if object has a given property
new	create instance
Typeof	check type of object

Ternary Operator:

- It helps to write an if else statement in a single line.
- Syntax: condition ? expressionIfTrue : expressionIfFalse;

```
let age = 18;

let message = age >= 18 ? "Adult" : "Minor";
console.log(message); // Adult

```

Type Conversion:

- Type Conversion in JavaScript means changing a value from one data type to another

Implicit:

- JavaScript automatically converts types when needed.
- Eg:

```
let result = "5" + 2;
console.log(result); // "52"

```

Explicit:

- You manually convert types using functions.
- Eg:

```
console.log(Number("10"));      // 10
console.log(Number("abc"));     // NaN
console.log(String(123));    // "123"
console.log((123).toString()); // "123"
console.log(Boolean(1));      // true
console.log(Boolean(0));      // false
console.log(Boolean(""));     // false
console.log(Boolean("hi"));   // true
```

Conditional Statements:

- Used to perform diff actions for diff decisions

1. If : Execute some code only if a specifiedcondition is true

```
if(condition)
{
    Code to execute
}
```

2. If else: Execute some code if the conditionis true and another code if the condition is false

```
if(condition)
{
    Code to execute
}
else {
    other code execute
}
```

3. Else if: Handling multiple possible conditions and outputs, evaluating more than two options based on whether the conditions are true or false.

```
if(condition)
{
    Code to execute}
else if {
    some other code executes
}
else {
    other code
}
```

4. Switch: Select one of many blocks of codeto be executed

```
let a=5
switch
(a) {
    case 1:
    console.log ("1 is executed")
    break
    case 2:
    console.log ("2 is executed")
    break
    case 3:
    console.log ("3 is executed”)
    break
    case 4:
    console.log ("4 is executed") break
    case 5:
    console.log ("5 is executed") break
    case 6:
    console.log ("6 is executed") break
}
```

Loops in Javascript:

- Same block of code to run over and over again in a row we use loops
- In JavaScript there are two different kind of loops:
  - for - loops through a block of code a specified number of times

```
for (initialization; condition; increment/decrement)
{
Statement to execute
}
```

```
1   for (var i = 0; i <= 10; i++) {
2       console.log(i)
3   }// 1 2 3 4 5 6 7 8 9 10
4
```

  - while - loops through a block of code while a specified condition is true

```
while (var<=end value)
{
    // code to be executed
}
```

```
1   var   i=0; while (i<=10)
2   {
3   console.log("The number is " + i);
4   i=i+1;
5   }// The number is 0 ........... The number is 10
```

  - Do while - This loop will always execute a block of code once, and then it will repeat the loop as long as the specified condition is true. This loop will always be executed at least once, even if the condition is false, because the code is executed before the condition is tested.

```
do
{
    //code to be executed
}while (var<=end value);
```

```
1   var i=0; do
2   {
3   console.log("The number is " + i);
4   i=i+1;
5   }
6   while (i<0);
```

Break and Continue Statements:

- There are two special statements that can be used inside loops: break and continue.

Break:

- The break command will break the loop and continue executing the code that follows afterthe loop (if any condition).

```
var i=0;
for (i=0;i<=10;i++)
{
if (i==3)
{
break;
}
console.log("The number is " + i);
}//The number is 0 The number is 1 The number is 2
```

Continue:

- The continue command will break the current loop and continue with the next value.

```
var i=0
for (i=0;i<=10;i++){
{
if (i==3)
    continue;
}
console.log("The number is " + i);
}//The number is 0 .... The number is 10
```

Functions in JavaScript:

- A function is a self-contained piece of code that performs a particular task.
- A function is a reusable code-block that will be executed by an event, or when the function is called.
- Function name should represent behaviors
- Wont get automatically invoked until we invoke it

Function Declaration:

- Syntax:

```
function func_name{
        //set of statement
} //function declaration

fun_name() //function call
```

```
1   function greet() {
2     console.log("Hello!");
3   }
4
5   greet();
```

Function Expression:

- Function stored inside a variable.

```
1   const greet = function () {
2     console.log("Hello!");
3   };
4
5   greet();
```

Parameters vs Arguments:

Parameters:

- A parameter is a variable listed in a function definition.
- It acts as a placeholder for the value that you pass into the function when calling it.

```
function greet(name) {  // "name" is a parameter
  console.log("Hello, " + name);
}

greet("Abc");
```

Types of Parameters in JavaScript:

1. **Default Parameters:**

- You can assign a default value if no argument is passed.

```
function greet(name = "Abc") {
  console.log("Hello, " + name);
}

greet(); // Hello, Abc
```

2. **Rest Parameters (...):**

- A function take multiple arguments and stores them in an array.

```
function sum(...numbers) {
  console.log(numbers);
}

sum(1, 2, 3, 4); //[1, 2, 3, 4]
```

3. **Destructured Parameters:**

- Extracting values from objects or arrays directly in the function parameters, instead of accessing them inside the function.

```
function displayUser({ name, age }) {
  console.log(name, age);//Abc 22
}

displayUser({ name: "Abc", age: 22 });
```

4. **Optional Parameters:**

- Function parameters are not strictly enforced, so if you call a function with fewer arguments than it expects, the missing parameters are automatically set to undefined

```
function greet(name, age) {
  console.log(name); // "Abc"
  console.log(age);  // undefined
}

greet("Abc");
```

Arguments:

- Arguments are the actual value passed to a function when it is called.

```
function add(a, b) {      // Parameters: a, b
  return a + b;
}

let result = add(5, 3);   // Arguments: 5, 3
console.log(result);      // 8
```

	Description	Example
Parameters	Variables listed in the function definition	function greet(name, age)
Arguments	Actual values passed when calling the function	greet(“Abc", 22)

Return Statement:

- The return statement is used to specify the value that is returned from the function. So,functions that are going to return a value must use the return statement.

```
function prod(a,b)
{
    x=a*b;
    return x;
}
res=prod(2,3)
console.log(res)//6
```

Types of functions:

1. Named functions:

- A function with a name, defined normally.
- It is reusable.

```
function greet() {
  console.log("Hello");
}

greet();
```

2. Parameterized functions:

- A function that accepts inputs (parameters).

```
function greet(name) { // parameter
  console.log("Hello " + name);
}

greet("Abc"); // argument
```

3. Return:

- Returns output using return
- It is the last executed statement
- Can return string, variable, number, boolean, console.log, array, objects

```
function add(a, b) {
  return a + b;
}

let result = add(2, 3);
console.log(result); // 5
```

4. Anonymous:

- Anonymous functions are function without a name.
- It is used in callbacks and expressions

```
const greet = function () {
  console.log("Hello");
};

greet();
```

5. Callback:

- A function passed as an argument to another function.

```
function add(a, b, callback) {
  let result = a + b;
  callback(result); // calling the callback
}
function display(result) {
  console.log("Result is: " + result);
}
add(2, 3, display);
```

6. Higher order functions:

- A higher order function is a function that takes one or more functions as arguments, or returns a function as its result.

```
function calculator(a, b, operation) {
  return operation(a, b);
}
function add(x, y) {
  return x + y;
}
console.log(calculator(2, 3, add)); // 5
```

7. Arrow function:

- Arrow function is a shorter way to write function using =>
- It is used for clearer and concise code
- Syntax:

```
const functionName = (parameters) => {
    // code
```

```
const add = (a, b) => {
  return a + b;
};

console.log(add(2,3))
```

};

8. IIFE's:

- IIFE is Immediately Invoked Function Expression
- It's a function that is defined and executed immediately after it's created.
- It avoids global scope pollution
- No function name is used, parentheses are used inside which an anonymous function is added

```
(function () {
  console.log("Runs immediately");
})();

```

Javascript Currying:

- Currying is a technique where a function with multiple arguments is transformed into a sequence of functions, each taking one argument at a time.

```
function add(a) {
    return function(b) {
        return a + b;
    }
}
const addTwo = add(5);  // First function call with 5
console.log(addTwo(4));
```

- The first function takes the first argument and gives back a new function to take the next one.
- The returned function takes the next argument and keeps going until all the arguments are given.
- Once all the arguments are provided, the final result is calculated and returned.

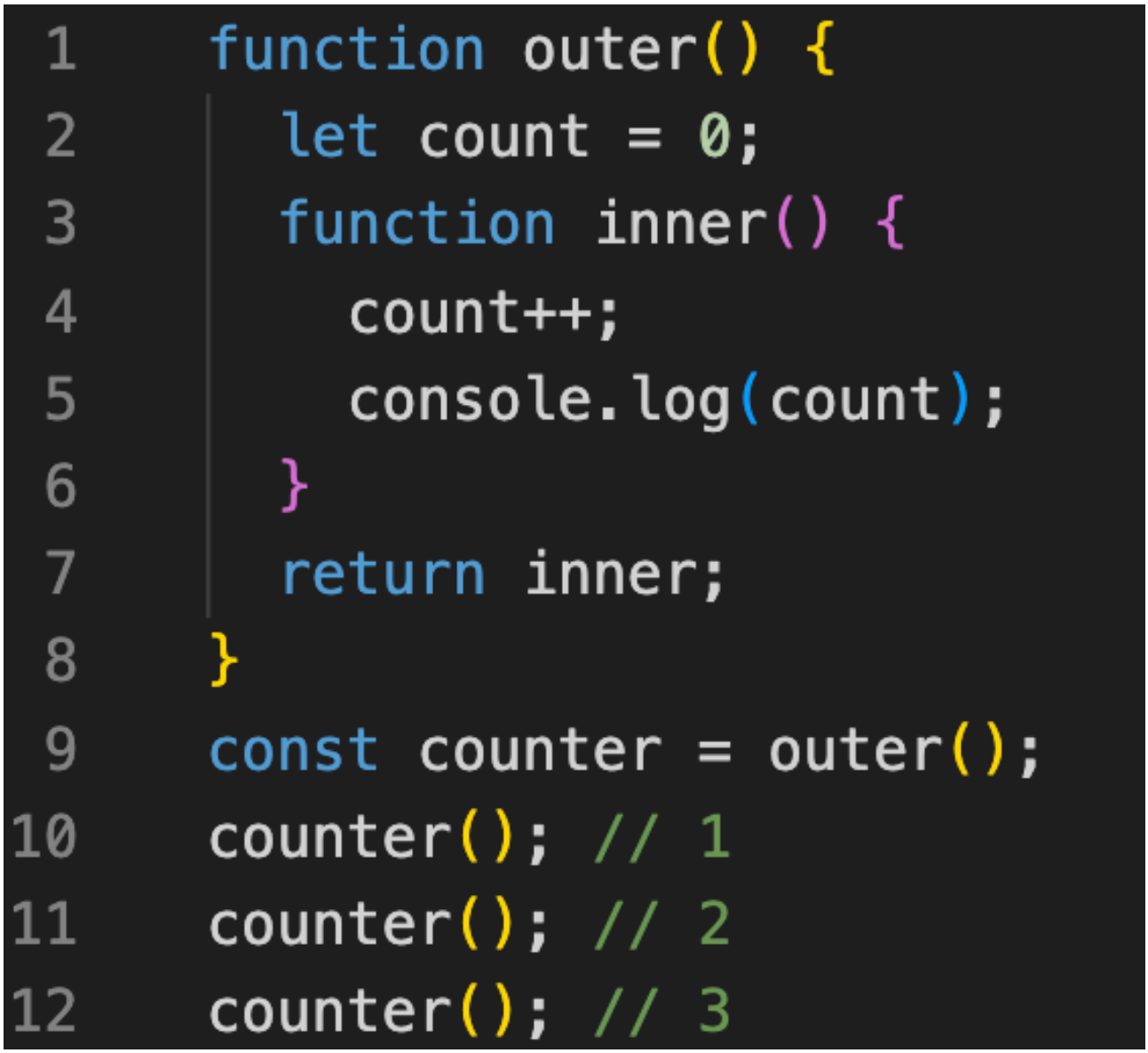
```
//currying with arrow function
const add = a => b => a + b;
console.log(add(5)(4));
```

```
function getAPIData(callback){
    console.log("Connecting to API")
    setTimeout(function(){
    console.log("Accessing data From API")
    let status="Success"
    callback(status)},4000)
}
function ShowAPIData(status){
    console.log("Status From API is "+status)
}
```

- Here getAPIData will execute first and then that will intern return the ShowAPIData method and return the status.

Javascript Closures:

- A closure in JavaScript is when a function remembers variables from its outer scope even after that outer function has finished executing.
- Closure is A function with the lexical environment in which it was created.



```
function outer() {
  let count = 0;
  function inner() {
    count++;
    console.log(count);
  }
  return inner;
}
const counter = outer();
counter(); // 1
counter(); // 2
counter(); // 3
```

Hoisting in JavaScript :

- Hoisting is the default behavior of moving all the declarations at the top of the scope beforecode execution.
- JavaScript only hoists declarations, not initializations.
- JavaScript allocates memory for all variables and functions defined in the program beforeexecution
- Function declarations are hoisted but function expressions are not hoisted.

```
sayHello() //output: Hi Good Morning
function sayHello()
{
console.log("Hi Good Morning")
}
```

```
console.log(add(10,20))//error add is not a function
var add=(a,b)=>{
return a+b
}
```

```
console.log(a)//undefined
var a=10;
```

Arrays:

- An array is a special variable, which can hold more than one value
- In Java Script arrays are heterogeneous means they can hold any type of variable.
- In Java Script arrays are dynamic in nature which means they can be dynamically updated

Creating Arrays:

- There are 2 ways to create a array:
    1. Using an array:
        - Syntax: const array_name = [item1, item2, …];

```
const array = [1, "Abc", true];
console.log(array)

const array2=[]
array2[0]=1;
array2[1]= "Abc";
array2[2]=true;
console.log(array2)
```

    2. Using new Keyword:

```
let numbers = new Array(1, 2, 3, 4);
console.log(numbers)
```

Accessing Elements:

- Array elements are accessed by their index, starting at 0.

```
let fruits = ['Apple', 'Banana', 'Cherry'];

console.log(fruits[0]); // 'Apple'
console.log(fruits[2]); // 'Cherry'
```

Array Methods:

- **length:**

The length property returns the length (size) of an array.

```
1    const array=[1,2,3,4,5,6,7]
2    console.log(array.length)//7
3
```

- **toString():**

The JavaScript method toString() converts an array to a string of (comma separated) array values

```
1    const array=[1,2,1,3,4,8,9]
2    let   a=array.toString()
3    console.log(a)//1,2,1,3,4,8,9
```

- **join():**

The join() method joins all array elements into a string.

It behaves just like toString(), but in addition you can specify the separator.

```
1    const array=[1,2,1,3,4,8,9]
2    let a=array.join(";")
3    console.log(a) // 1;2;1;3;4;8;9
4
```

- **pop():**

The pop() method removes the last element from an array.

It returns the value that was popped out.

```
1    const array=[1,2,1,3,4,8,15]
2    let a=array.pop()
3    console.log(a)// 15
4    console.log(array)// [ 1, 2, 1, 3, 4, 8 ]
5
```

- **push():**

The push() method adds a new element to an array at the end.

It returns the new length of array.

```
const array=[1,2,1,3,4,8]
let a=array.push(100)
console.log(a)//7
console.log(array)// [ 1, 2, 1, 3, 4, 8, 100 ]
```

- **shift():**

The shift() method removes the first array element and "shifts" all other elements to a lowerindex.

The shift() method returns the value that was removed.

```
const array=[1,2,1,3,4,8]
let a=array.shift()
console.log(a) //1
console.log(array)//[ 2, 1, 3, 4, 8 ]
```

- **unshift():**

It adds the element at first index.

The method returns the new array length

```
const array=[1,2,1,3,4,8]
let a=array.unshift(10)
console.log(a) //7
console.log(array)//[ 10,1,2, 1, 3, 4, 8 ]
```

- **slice():**

The slice(start index, end index) method slices out a piece of an array into a new array.

The slice() method creates a new array and does not remove any elements from the source array.

```
const array=[10,20,11,23,42,87,34]
let a=array.slice(2,6)
console.log(a)//[ 11, 23, 42, 87 ]
```

- **splice():**

array.splice(start, deleteCount, item1, item2, …) method used to add, remove, or replace elements directly in an array.
start: index where changes begin
deleteCount: number of elements to remove
item1, item2…: elements to add

```
const arr = [1, 2, 3, 4];

arr.splice(1, 2); // remove 2 elements starting from index 1

console.log(arr); // [1, 4]
```

```
const arr = [1, 2, 4];

arr.splice(2, 0, 3); // add 3 at index 2

console.log(arr); // [1, 2, 3, 4]
```

```
const arr = [1, 2, 3];

arr.splice(1, 1, 99); // replace 2 with 99

console.log(arr); // [1, 99, 3]
```

- **indexof:**

indexOf(value) Returns the first index of the value, or -1 if not found

- **includes:**

includes(value) Returns true if the value exists in the array, otherwise false

```
let array = ['A', 1, 3.4];

console.log(array.indexOf(1)); // 1
console.log(array.includes('M')); // false
```

Looping Arrays:

i. for loop:

```
let array = [12, 56, 83, 56, 45, 32];

for (let i = 0; i < array.length; i++) {
  console.log(array[i]);
}
```

ii. for...of loop:

```
let array = [12, 56, 83, 56, 45, 32];

for (let arr of array) {
  console.log(arr);
}
```

iii. forEach method:

Executes a function for each element.

```
let array = [12, 56, 83, 56, 45, 32];

array.forEach((arr, index) => {
  console.log(`${index}: ${arr}`);
});
```

iv. map():
Creates a **new array** by applying a function to each element

```
let numbers = [1, 2, 3, 4, 5];
let squares = numbers.map(n => n * n); // [1, 4, 9, 16, 25]
console.log(squares)

```

v. filter:
Creates a new array with elements that satisfy a condition

```
let numbers = [1, 2, 3, 4, 5];
let even = numbers.filter(n => n % 2 === 0); // [2, 4]
console.log(even)
```

vi. reduce:
Reduces the array to a single value using a callback

```
let numbers = [1, 2, 3, 4, 5];
let sum = numbers.reduce((acc, curr) => acc + curr, 0);//15
console.log(sum)
```

What are Objects?

- Objects are used to store the data in key-value pair.
- Eg: const person = {

  firstName : "Abc",

  lastName : "S",

  age : 23,

  height : 6.0

  };

Object Properties & Methods:

- Object properties: They are variables inside objects
- Methods: They are functions inside objects

```
let person = {
  name: "Abc",
  age: 20,

  // Method
  greet: function() {
    console.log("Hello!");
  }
};
console.log(person.name); // Property
person.greet(); // Method
```

Accessing Properties:

- You can access object data in two main ways:

1. Dot Notation:

```
let person = {
  name: "Abc",
  age: 20
};

console.log(person.name); // Abc
console.log(person.age);  // 20
```

- Cannot use dynamic keys

2. Bracket Notation:
   - Useful when Property name is dynamic, Property has spaces

```
let person = {
  name: "Abc",
  age: 20
};

console.log(person["name"]); // Abc

let key = "age";
console.log(person[key]); // 20
```

Adding New Properties

You can add new properties to an existing object by simply giving it a value.If property is already existing, then that property will be overridden.

person.nationality = “India”;

Deleting Properties

- The delete keyword deletes a property from an object.
- The delete keyword deletes both the value of the property and the property itself.

```
const person = {
firstName : "Abc",
lastName : "S S",
age : 20,
height : 5.7
};
delete person.age
console.log(person)
```

Nested Objects:
Objects can contain other objects inside them that is a object inside another object

```
let user = {
  name: "Abc",
  address: {
    city: "Bangalore",
    zip: 560001
  }
};

console.log(user.address.city); // Bangalore
```

Array Of Objects:

We can store an objects inside the array.

```
let obj=[{
    name:'Abc',
    age:23,
    place:'Bangalore',
    Height:5.7
},{
    name:'Def',
    age:24,
    place:'Delhi',
    Height:5.9
},{
    name:'Ghi',
    age:29,
    place:'Bangalore',
    Height:5.7
},{
    name:'Jkl',
    age:20,
    place:'Bangalore',
    Height:6.0
}]
```

We can access the individual objects according to the indexes.

We can access the object property using the index of object and key of object.

Console.log(obj[0])//print object at 0 index.

Strings:

String Creation:

- You can create strings using single quotes, double quotes, template literals (backticks)

```
let str1 = 'Hello';
let str2 = "World";
let name = "Abc";
let str3 = `Hello ${name}`;
console.log(str3)// Hello Abc
```

length:

Returns the number of characters.

```
let text = "Hello";
console.log(text.length); // 5
```

toUpperCase() :

convert text to upper case.

```
let name1= "Abcd A"
console.log(name1.toUpperCase())//ABCD A
```

toLowerCase() :

convert text to lower case.

```
let name1= "Abcd A"
console.log(name1.toLowerCase())//abcd a
```

trim():

The trim() method removes whitespace from both sides of a string

```
let name1= "      Abcd A     "
console.log(name1.trim())//Abcd A

```

trimStart():

The trimStart() method will removes whitespace only from the start of a string

```
let name1= "      Abcd A     "
console.log(name1.trimStart())//"Abcd A     "

```

trimEnd():

The trimEnd() method will removes whitespace only from the Ending of a string.

```
let name1= "      Abcd A     "
console.log(name1.trimEnd())// "     Abcd A"

```

substring():

substring() is similar to slice(). The difference is that start and end values will not take negative index it will be treated as 0.

```
let name1= "Abcd A"
console.log(name1.substring(2,7))// cd A
```

replace():

This method is used to replace a string with another String

```
let name1= "Abcd A"
console.log(name1.replace("Abcd","Rahul"))//Rahul A

```

includes():

Checks if a string contains a value.

```
let text = "Hello World";
console.log(text.includes("World")); // true
console.log(text.includes("JavaScript"));  // false

```

charAt():

The charAt() method returns the character at a specified index in a string.

```
let name= "Rahul S"
console.log(name.charAt(4))//l

```

charCodeAt():

The charCodeAt() method returns the code of the character at a specified index in a string.

```
let name= "Rahul S"
console.log(name.charCodeAt(4))//108
```

slice():

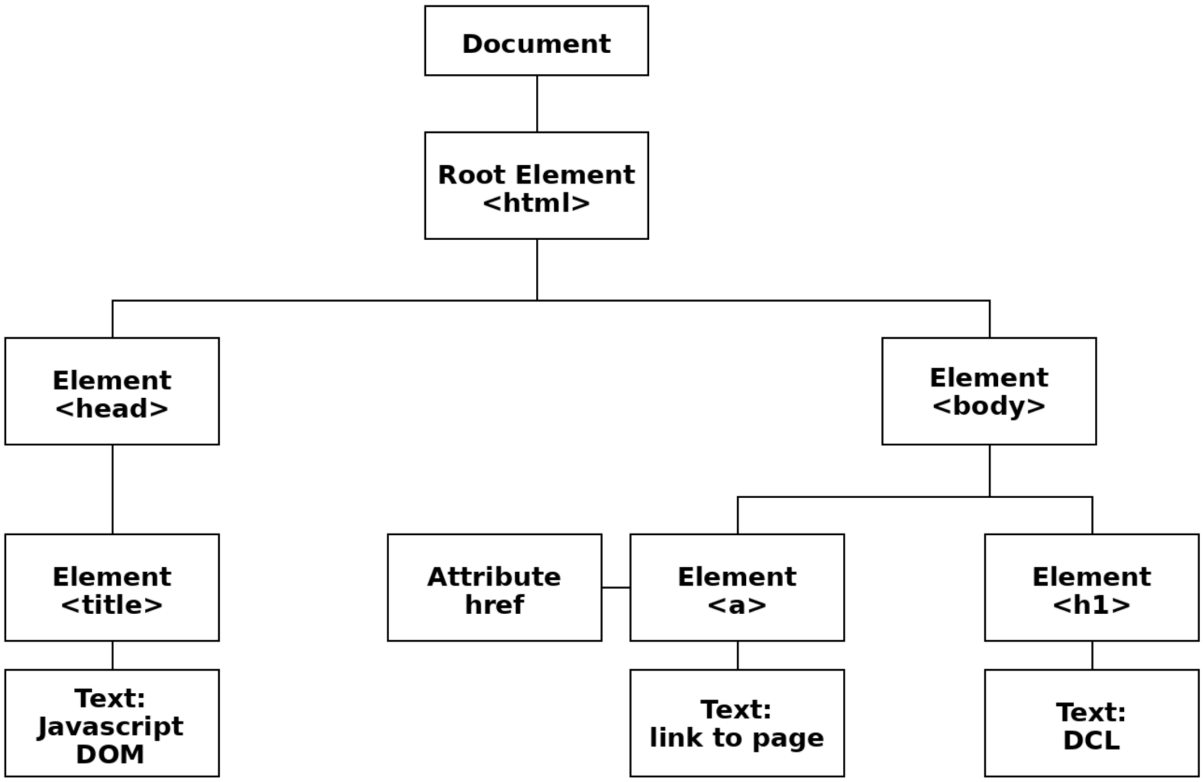
slice() extracts a part of a string and returns the extracted part in a new string.

The method takes 2 parameters ie start and end position.

```
let name1= "Abcd A"
console.log(name1.slice(2,7))//cd A

let name2= "Defghe S"
console.log(name2.slice(-6,-1))//efghe

```

DOM(Document Object Model):

- DOM (Document Object Model) is a programming interface for web pages.
- The HTML DOM model is constructed as a tree of Objects with the document as the parentelement.
- With the help of DOM, we can access and manipulate the html element in JavaScript.

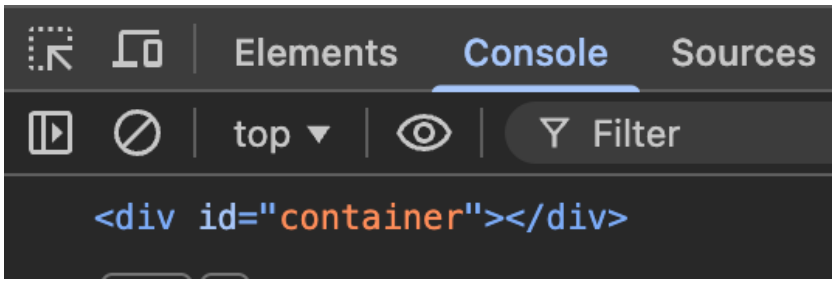


Selecting Elements:

1. getElementById :

- Selects a single element by ID

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <div id="container"></div>
    <script>
        let element=document.getElementById('container')
        console.log(element);
    </script>
</body>
</html>
```



2. getElementsByClassName:

- Find all HTML elements with the same class name
- Syntax: document.getElementsByClassName("className")
- If multiple elements have the same class, all are returned

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
</head>
<body>
<div id="container" class="divs"></div>
<script>
let element=document.getElementsByClassName('divs')
console.log(element)
</script>
</body>
</html>

```

```
▸ HTMLCollection [div#container.divs, container: div#container.divs]
> cmd i to turn on code suggestions. Don't show again NEW
```

3. querySelector:

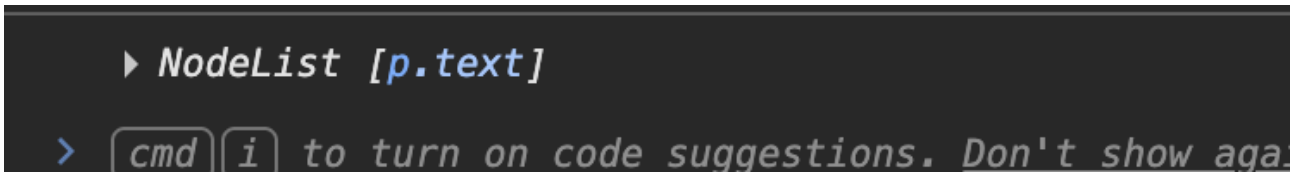
- It selects the first matching element.
- If you want to find all HTML elements that matches a specified CSS selector (id, classnames, types,), use the querySelector() method.

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5   </head>
6   <body>
7   <div id="container" class="divs"></div>
8   <script>
9   let element=document. querySelector('.divs')
10  console.log(element)
11  </script>
12  </body>
13  </html>
```

4. querySelectorAll:

- It selects all matching elements.
- Returns a NodeList can loop through it easily

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5   </head>
6   <body>
7   <p class="text">One</p>
8   <script>
9   let element = document.querySelectorAll(".text");
10  console.log(element);
11  </script>
12  </body>
13  </html>
14
```



Manipulating Elements

Text:

To change or get text of an element we use:

innerText:

- It is used to add the text content inside the element
- Gives visible text.

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="container">Original Text</div>
    <button onclick="changeContent()">Click Me</button>
    <script>
        function changeContent() {
            // Add text
            document.getElementById('container').innerText = 'Text added in DOM';
        }
    </script>

</body>
</html>

```

innerHTML:

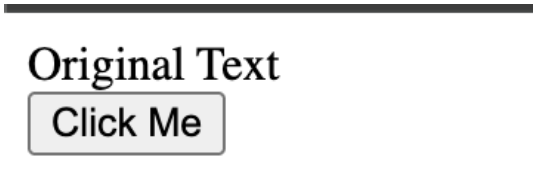
- It is used to add the html content inside the element.

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="container">Original Content</div>
    <button onclick="addHeading()">Click Me</button>

    <script>
        function addHeading() {
            document.getElementById('container').innerHTML = '<h1>Heading 1 created in DOM</h1>';
        }
    </script>
</body>
</html>
```

style:

- It is used to add the style for the element:
- Respects CSS visibility to some extent
- Here we need to write the properties in camelCase Convention and the value need to be specified in string format.

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="container">Original Text</div>
    <button onclick="changeContent()">Click Me</button>

    <script>
        function changeContent() {
            // Add text
            document.getElementById('container').innerText = 'Text added in DOM';
            // Add style
            document.getElementById('container').style.backgroundColor = 'blue';
        }
    </script>
</body>
</html>
```



createElement:

- This method is used to create a html element.
- Eg: const heading= document.createElement('h1')

appendChild:

- This method is used to add the element created to html document.

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>

    <div id="container">
        <p>Old content</p>
    </div>

    <script>
        const heading = document.createElement('h1');
        heading.innerText = "Heading created using createElement";
        document.getElementById('container').appendChild(heading);
    </script>

</body>
</html>
</body>
</html>
```

remove:

- It is used to remove element from the document

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>

<div id="container">
    <h1 id="heading">Hello</h1>
</div>

<script>
    let el = document.getElementById('heading');
    el.remove();  // removes <h1>Hello</h1>
</script>

</body>
</html>
</body>
</html>
```

setAttribute:

- It is used to add an attribute for the element

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="container"></div>

    <script>
        let heading = document.createElement('h1');
        heading.setAttribute('class', 'h1');

        let heading2 = document.createElement('h1');
        heading2.setAttribute('id', 'headings');

        heading.innerText = "Heading with class";
        heading2.innerText = "Heading with id";

        document.getElementById('container').appendChild(heading);
        document.getElementById('container').appendChild(heading2);
    </script>

</body>
</html>
```

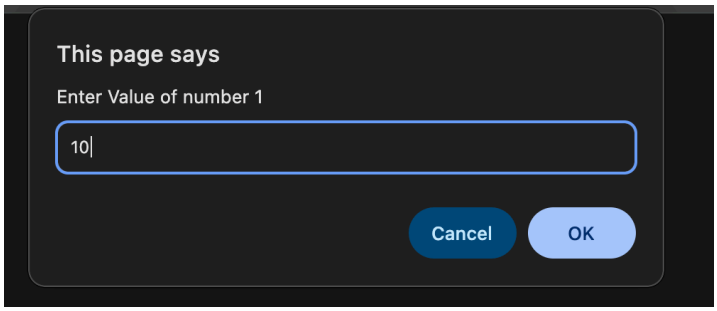
removeAttribute:

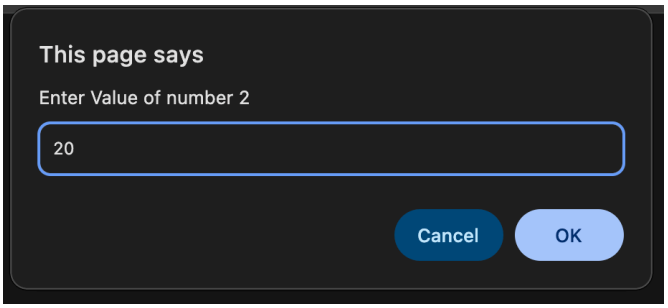
- It is used to remove attribute from the element

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>removeAttribute Example</title>
</head>
<body>

    <div id="container">
        This is a container
    </div>
    <script>
        let div = document.getElementById('container');
        div.removeAttribute('id');
    </script>
</body>
</html>
```

Program to take input from the user and print the sum in htmldocument using DOM

```
1   <!DOCTYPE html>
2   <html>
3   <head>
4   <title>DOM DEMO</title>
5   </head>
6   <body>
7   <p> The Sum is <span id="sum"> </span></p>
8   <script>
9   let num1=Number(prompt("Enter Value of number 1"))
10  let num2=Number(prompt("Enter Value of number 2"))
11
12  let sum=num1+num2
13
14  document.getElementById('sum').innerText=sum
15  </script>
16  </body>
17  </html>
```





The Sum is 30

Events in JavaScript:

- Event are the action or occurrence that happen in the web browser such as click, keypress,form submission, mouse hover.
- JavaScript provide a build in mechanism for handling the events, allow you to create aninteractive web application.

Event Handling:

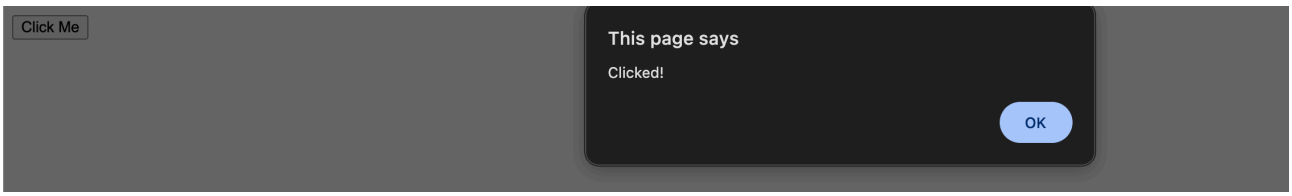
- Event handling means responding to user actions like click, keypress, mouse movement, etc.

Event	Event Handler
click	onclick
mouseover	onmouseover
mouseout	onmouseout
mousedown	onmousedown
keyup	onkeyup
keydown	onkeydown
Focus	onfocus
Submit	onsubmit
onload	onload

Click:

- onclick is an event handler
- It runs when the button is clicked
- Only one function can be assigned

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>

    <button id="btn">Click Me</button>
    <script>
        let button = document.getElementById("btn");

        button.onclick = function () {
            alert("Clicked!");
        };
    </script>
</body>
</html>
```

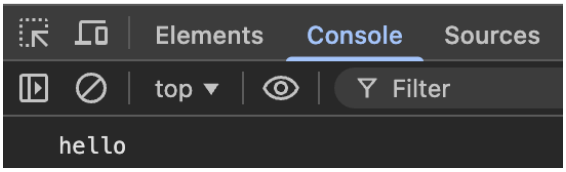


Change:

- Triggered when the value of an input changes AND loses focus
- Input field loses focus after change
- Dropdown selection changes

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>

    <input type="text" id="input" placeholder="Type something">
    <script>
        let input = document.getElementById("input");
        input.onchange = function () {
            console.log(input.value);
        };
    </script>
</body>
</html>

```

hello

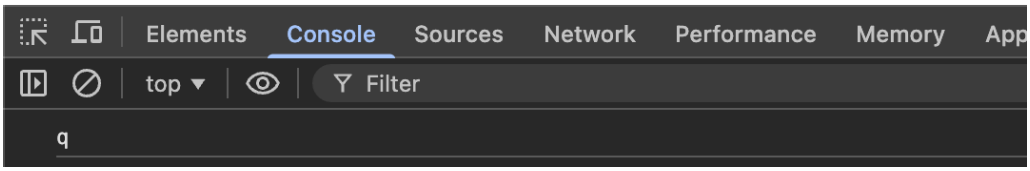


keypress:

- Triggered when a key is pressed on keyboard
- `keypress` is deprecated in modern JS instead use `keydown` (recommended) Or keyup

```html
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <h2>Press any key...</h2>
    <script>
        document.addEventListener("keypress", function (e) {
            console.log(e.key);
        });
    </script>
</body>
</html>
```

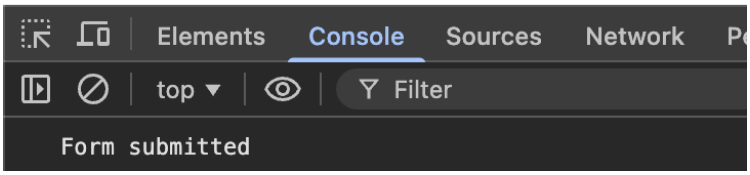
Press any key...



Submit:

- Triggered when a form is submitted

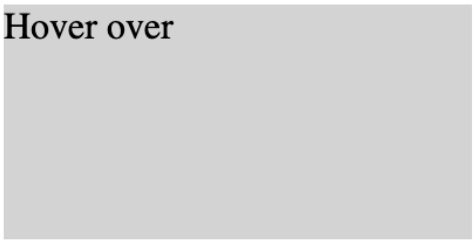
```html
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <form id="myForm">
        <input type="text" placeholder="Enter name">
        <button type="submit">Submit</button>
    </form>
    <script>
        let form = document.getElementById("myForm");
        form.addEventListener("submit", function (e) {
            e.preventDefault(); // stop page reload
            console.log("Form submitted");
        });
    </script>

</body>
</html>
```



mouseover:

- Triggered when the mouse comes over an element
- Color changes but does NOT go back

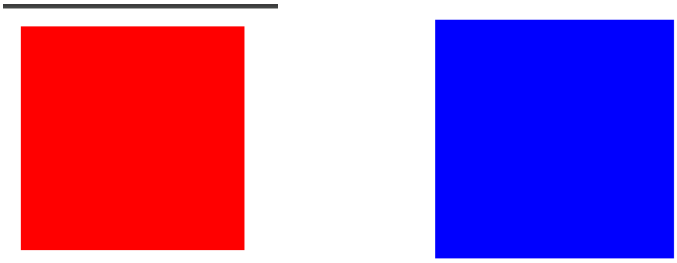
```html
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="box" style="width:200px; height:100px; background-color:lightgray;">
        Hover over
    </div>
    <script>
        let box = document.getElementById("box");
        box.addEventListener("mouseover", function () {
            box.style.backgroundColor = "yellow";
        });
    </script>
</body>
</html>
```



mouseout:

- Triggered when the mouse leaves an element

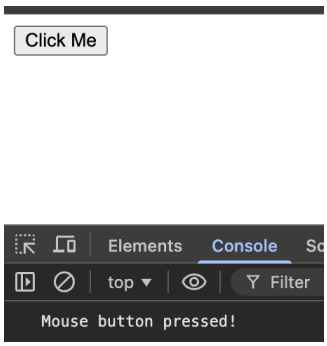
```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
    <div id="box" style="width:100px;height:100px;background:red;"></div>

    <script>
        let box = document.getElementById("box");
        box.addEventListener("mouseout", function () {
            box.style.backgroundColor = "blue";
            });
    </script>
</body>
</html>
```



mousedown:

- Triggered when a mouse button is pressed down.

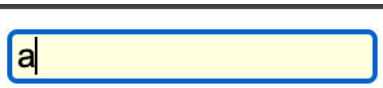
```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
<button id="btn">Click Me</button>
<script>
let btn = document.getElementById("btn");
btn.addEventListener("mousedown", function () {
    console.log("Mouse button pressed!");
});
</script>
</body>
</html>
```



Focus:

- Triggered when an input field gets focus (clicked or tabbed into)

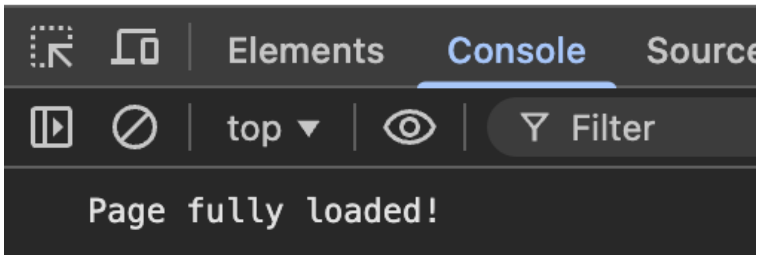
```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
<input type="text" id="name">

<script>
let input = document.getElementById("name");

input.addEventListener("focus", function () {
    input.style.backgroundColor = "lightyellow";
});
</script>
</body>
</html>
```



onload:

- Triggered when the page fully loads

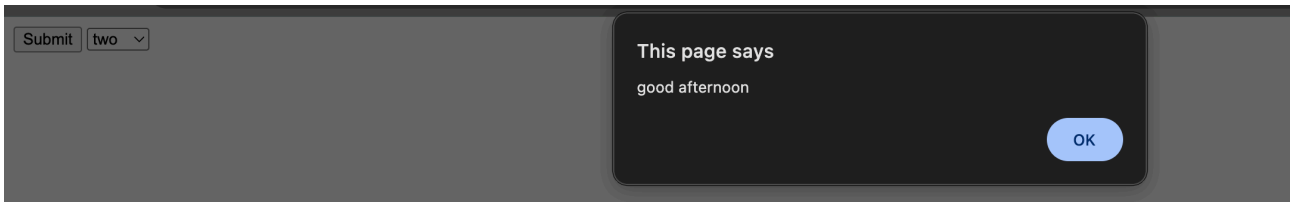
```html
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
<script>
window.onload = function () {
    console.log("Page fully loaded!");
};
</script>
</body>
</html>
```

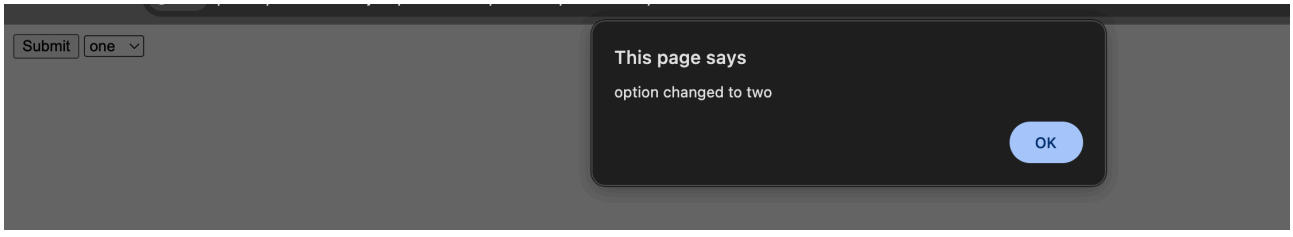


Event Litesner:
•This function that wait for the event to occur and response to it.
•Event listener listen for the event and response for the event by calling a function.

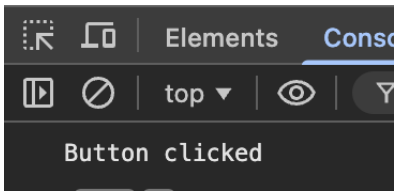
addEventListener():
• Used to attach an event to an element
• This function is an example of higher order function which will take event as one argumentas a string and a callback function which will affect when the event occurs.
• Syntax: element.addEventListener(event, function);

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <button id="btn">Submit</button>
    <select id="dropdown">
        <option value="one">one</option>
        <option value="two">two</option>
        <option value="three">three</option>
        <option value="four">four</option>
    </select>
    <script>
        let btnref=document.getElementById('btn')
        btnref.addEventListener('click',()=>{
            alert("good afternoon")
        })
        let select=document.getElementById('dropdown')
        select.addEventListener('change',(event)=>{
            alert("option changed to " +event.target.value)
        })
    </script>
</body>
</html>
```

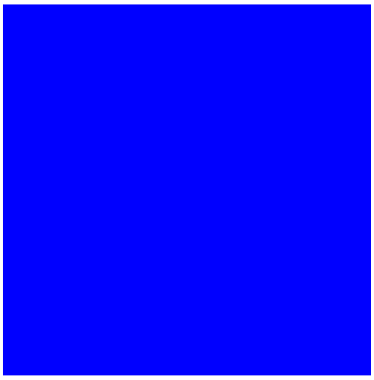




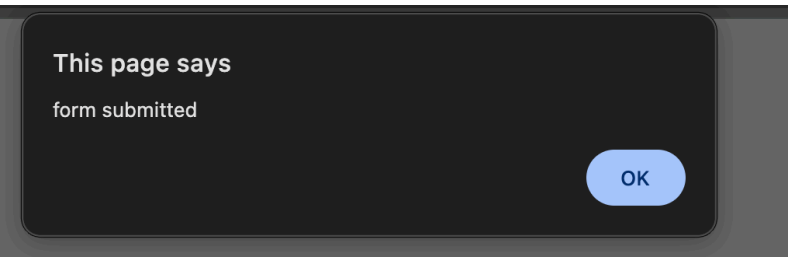
```html
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body>
<button id="btn">Click Me</button>

<script>
let btn = document.getElementById("btn");

btn.addEventListener("click", function () {
    console.log("Button clicked");
});
</script>
</body>
</html>
```

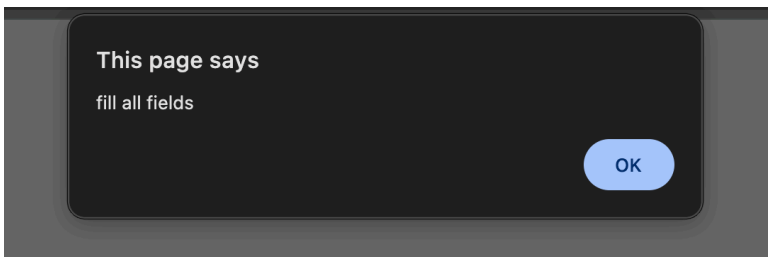


```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        .one{
            background-color: blue;
            height: 300px;
            width: 300px;
            margin: 50px auto;
        }
    </style>
</head>
<body>
    <div class="one" id="one" onmouseover="Mouseover()" onmouseout="Mouseout()"></div>
    <script>
        function Mouseover(){
            let divref=document.getElementById('one')
            divref.style.backgroundColor="red"
            divref.style.borderRadius="0%"
        }

        function Mouseout(){
            let divref=document.getElementById('one')
            divref.style.backgroundColor="yellow"
            divref.style.borderRadius="50%"
        }
    </script>
</body>
</html>
```



```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <center>
        <form onsubmit="FormValidation()">
            <label>username</label>
            <input type="text" name="username" id="username"><br><br>
            <label>password</label>
            <input type="password" name="password" id="password"><br><br>
            <input type="submit">
        </form>
    </center>
    <script>
        function FormValidation(){
            let username=document.getElementById('username').value
            let password=document.getElementById('password').value
            if (username =="" || password==""){
                alert("fill all fields")
            }
            else{
                alert("form submitted")
            }
        }
    </script>
</body>
</html>
```



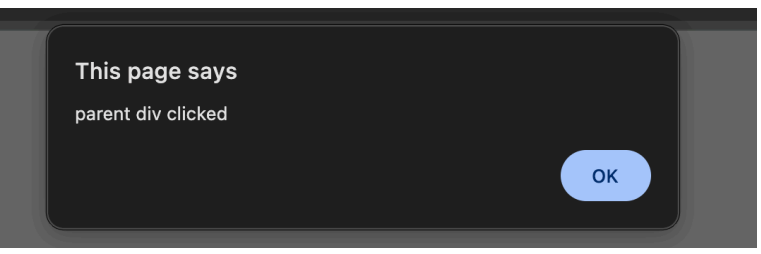


```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
</head>
<body>
    <input type="text" placeholder="click or tab into me" id="one" onfocus="FocusOnElement()">
    <script>
        function FocusOnElement(){
            let input=document.getElementById("one")
            input.style.border="2px solid green"
            input.style.outline="none"
            input.style.backgroundColor="pink"
            input.style.height="50px"
            input.style.width="200px"
            input.style.boxShadow="0px 0px 10px blue"
        }
    </script>
</body>
</html>

```

Event capturing:

- Event flows from parent to child
- Syntax: addEventListener("click", function, true)
- In *capturing* the outer most element's event is handled first and then the inner

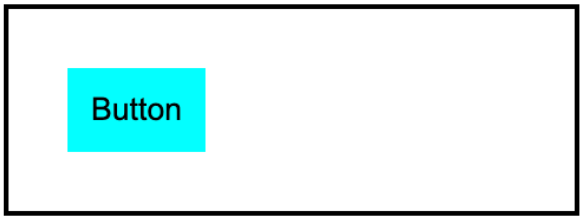
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        #parent{
            border: 2px solid black;
            padding:20px;
            width: 200px;
        }
        #child{
            margin: 5px;
            padding: 10px;
            background-color: aqua;
            border: none;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <div id="parent">
        <button id="child">Button</button>
    </div>
    <script>
        document.getElementById("parent").addEventListener("click",function(){
            alert("parent div clicked")
        },true);
        document.getElementById("child").addEventListener("click",function(){
            alert("child button clicked")
        });

    </script>
</body>
</html>
```



Event Bubbling:

- Event flows from child to parent
- In bubbling the inner most element's event is handled first and then the outer

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <style>
        #parent{
            border: 2px solid black;
            padding:20px;
            width: 200px;
        }
        #child{
            margin: 5px;
            padding: 10px;
            background-color: aqua;
            border: none;
            cursor: pointer;
        }
    </style>
</head>
<body>
    <div id="parent">
        <button id="child">Button</button>
    </div>
    <script>
        document.getElementById("parent").addEventListener("click",function(){
            alert("parent div clicked")
        });
        document.getElementById("child").addEventListener("click",function(){
            // event.stopPropagation(); to stop event bubbling
            alert("child button clicked")
        });

    </script>
</body>
</html>
```



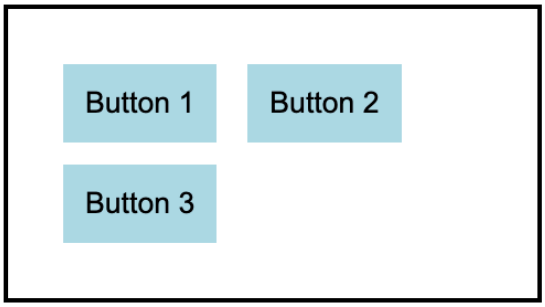
Event Delegation:

- A event delegation is where a parent is used to handle events of multiple child elements
- Reduces the number of event listeners.It uses event bubbling to capture events.

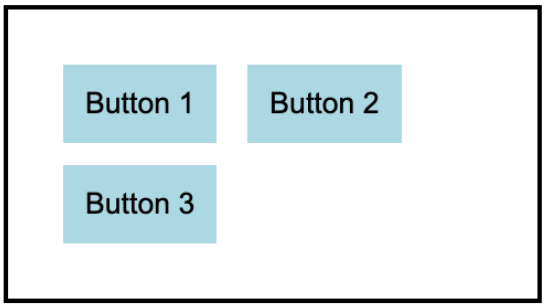
```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <style>
        #parent, #parent1{
            border:2px solid black;
            padding:20px;
            width:200px;
        }
        .child{
            margin:5px;
            padding:10px;
            background-color:lightblue;
            border:none;
            cursor:pointer;
        }
        </style>
</head>
```

```
<body>
    <h1>example without event delegation</h1>
    <div id="parent">
        <button class="child" onclick="Notify()">Button 1</button>
        <button class="child" onclick="Notify()">Button 2</button>
        <button class="child" onclick="Notify()">Button 3</button>
    </div>

    <h1> example with event delegation</h1>
    <div id="parent1" >
        <button class="child" >Button 1</button>
        <button class="child" >Button 2</button>
        <button class="child" >Button 3</button>
    </div>
    <script>
    function Notify(){
        alert("Button Clicked");
    }
    let parent1=document.getElementById("parent1")
    parent1.addEventListener("click",function(e){
    if(e.target && e.target.classList.contains("child")){
        alert("Clicked on "+ e.target.innerText);
    }
    })
    </script>
</body>
</html>
```

example without event delegation



example with event delegation



this:
this refers to the object that is executing the current function
Its value depends on how the function is called

```
let user = {
    name: "Abc",
    age: 22,
    greet: function () {
        console.log(this.name);//Abc
    }
};
user.greet();

```

```
const person = {
    name: "ABC",
    age: 30,
    greet() {
        console.log('Hello, my name is ' +
            this.name + ' and I am '
            + this.age +
            ' years old.');
    }
};

person.greet();
```

Template Literals

- It is used for string formatting with variables
- Use backticks (``) Introduced in ES6, they support easy interpolation and multi-line strings.

```
let x = 5;
let y = 10;
console.log(`The sum of ${x} and ${y} is ${x + y}`); //The sum of 5 and 10 is 15

```

Spread Operator(...)

- The spread operator is used to expand elements of arrays or objects.
- It is used for cloning arrays/objects, merging data, passing values
- It expands elements of arrays and strings or properties of objects into individual values.

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1];
console.log(arr2)//[ 1, 2, 3 ]

let obj1 = {name: "abc"};
let obj2 = {...obj1};
console.log(obj2)//{name: "abc"}

let a = [1, 2];
let b = [3, 4];
let result = [...a, ...b];
console.log(result)//[ 1, 2, 3, 4 ]

let arr = [1, 2];
let newArr = [...arr, 3];
console.log(newArr)//[ 1, 2, 3 ]
```

Rest Operator (...)

- The rest operator collects multiple values or properties into a single array.
- The rest parameter syntax allows a function to accept an indefinite number of arguments as an array.

```
function sum(...numbers) {
    return numbers.reduce((a, b) => a + b);
}

console.log(sum(1,2,3,4)); // 10
```

Destructuring:

• Destructuring is used to extract values from arrays or objects into variables.
• Array destructuring:Array members can be unpacked into different variables.

```
let arr = [10, 20, 30];
let [a, b, c] = arr;

console.log(a); // 10
```

• Object destructuring: Object destructuring means extracting values from objects into variables

```
let user = {
    name: "Abc",
    age: 22
};
let { name, age } = user;

console.log(name); // Abc
console.log(age);  // 22
```

Default Parameters:

• It is used to give the default values to the arguments, if no parameter is provided in the function call.

```
function greet(name = "Abc") {
    console.log(`Hello ${name}`);
}

greet(); // Hello Abc
greet("Ram"); // Hello Ram
```

```
function add(a = 0, b = 0) {
    return a + b;
}

console.log(add(1,2))// 3
console.log(add())// 0

```

Modules:

• Modules allow you to split your code into multiple files and reuse them.
• It helps in code organization, reusability, maintainability

Synchronous (Blocking):
• Code runs line by line, one after another.
• Next task waits until current task finishes.

```
console.log("Hi");
console.log("How are you?");
console.log("1");
console.log("2");
```

Asynchronous (Non-blocking):

- It allows multiple tasks to run independently of each other. In asynchronous code, a task can be initiated, and while waiting for it to complete, other tasks can proceed.

```
console.log("Hi");

setTimeout(() => {
    console.log("Geek");
}, 2000);

console.log("End");

```

- The code does is first it logs in Hi then rather than executing the setTimeout function it logs in End and then it runs the setTimeout function.

Callbacks:

- A callback is a function passed as an argument to another function and is invoked after the execution of main function.

```
function calculate(a, b, operation) {
    return operation(a, b);
}
function add(x, y) {
    return x + y;
}

console.log(calculate(2, 3, add));        // 5

```

Callback Hell:

• Callback hell is used to describe the nested callback stacked over bellow one another formatting a pyramid structure.

• Every callback depends/wait for previous callback.

• It is hard to understand and maintain

• To solve this we can use promises or async/await

Promises:

• Promise is an object representing the eventual completion or failure of an asynchronous operation

• A promise object can be in any 3 states

1. Pending : operation started (not finished)
2. Rejected: operation failed
3. fulfilled: operation completed

async / await:

• async: declare a function or method as asynchronous and can pause its execution to wait forcompletion of other process.

• await: make a suspension point where execution may wait for the result of async function ormethods.

```
async function fetchData() {
  try {
    let response = await fetch('url');
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error fetching data:', error);
  }
}
fetchData()

```

setTimeout:

• Executes a function after a delay (in milliseconds)

• Syntax: setTimeout(function, delay);

```
setTimeout(() => {
    console.log("Hello after 2 sec");
}, 2000);

```

setInterval:
- Executes a function repeatedly at a fixed interval
- Syntax: setInterval(function, delay);

```
let id = setInterval(() => {
    console.log("Runs every 1 sec");
}, 1000);

```

Debounceing :
- Debouncing ensures a function runs only after a certain delay after the last event or function has occurred
- Eg: Form validation, Search bar typing
- Debouncing works by clearing the previous timer, then starting a new timer and the functions

```
function debounce(func, delay) {
    let timer;

    return function (...args) {
        clearTimeout(timer);

        timer = setTimeout(() => {
            func.apply(this, args);
        }, delay);
    };
}
const handleSearch = debounce((value) => {
    console.log("Searching for:", value);
}, 500);

document.getElementById("search").addEventListener("input", (e) => {
    handleSearch(e.target.value);
});
```

runs only once after the delay

Throttling:
- Throttling ensures a function runs at most once in a given time interval.
- It limits how often a function runs.
- Eg: Scroll events, Window resize
- Function runs immediately and then ignores calls until time limit is over

```
function throttle(func, limit) {
    let lastCall = 0;

    return function (...args) {
        let now = Date.now();

        if (now - lastCall >= limit) {
            lastCall = now;
            func.apply(this, args);
        }
    };
}

const handleScroll = throttle(() => {
    console.log("Scrolling...");
}, 1000);

window.addEventListener("scroll", handleScroll);

```

Browser APIs:
Browser APIs are built-in features provided by the browser that allow JavaScript to interact with:

- Storage
- Network requests
- Data formats

Local Storage:

- Stores data with no expiration time.
- Even after browser is closed, after system restart the stored data does not expire until manually cleared
- Used for saving user preferences (dark mode) saving login token (careful with security) cart items in e-commerce remembering theme/language

Session Storage:

- Stores data only for one browser session.
- Data gets deleted when tab is closed, when browser is closed
- Used for otp verification sessions one-time page visit data multi-step form data

Feature	Local Storage	Session Storage
Storage Limit	~5MB	~5MB
Expiry	No expiry	Ends when tab closes
Accessible in	All tabs (same origin)	Only same tab
Use Case	Long-term storage	Temporary storage
Data Persistence	Yes	No

JSON Parsing:

• JSON is a string format.JS works with objects so we convert between them

• Object to JSON:

```
let obj = {name:"Abc", age:21};

let json = JSON.stringify(obj);

console.log(json);// {"name":"Abc","age":21}

```

• JSON to Object:

```
let json = '{"name":"Abc","age":21}';

let obj = JSON.parse(json);

console.log(obj.name); // Abc
```